Weekly Project Progress-3

Group Work

Oussema Kirmani – 5153785

Gurpreet Singh Galsian– 5154655

Dhruv Dewan – 5150667

Data Analysis with Python

Algoma University

COSC-5806

Zamilur Rahman

March 22, 2026

In this phase of the project, machine learning models were developed to predict heart disease based on patient health data. The dataset was preprocessed, balanced, and split into training and testing sets. Logistic Regression and Decision Tree models were implemented and evaluated.

Data Preprocessing

The "Heart Disease" column was cleaned and converted into numerical format:

```
df["Heart Disease"] = df["Heart Disease"].astype(str).str.strip()
df["Heart Disease"] = df["Heart Disease"].map({"Absence": 0, "Presence": 1})
```

Explanation

- Removed extra spaces from text values
- Converted categorical values into numerical form
- Absence → 0
- Presence → 1

	id	Age	Sex	Chest pain type	BP	Cholesterol	FBS over 120	EKG results	Max HR	Exercise angina	ST depression	Slope of ST	Number of vessels fluro	Thallium	Heart Disease
0	0	58	1	4	152	239	0	0	158	1	3.6	2	2	7	1
1	1	52	1	1	125	325	0	2	171	0	0.0	1	0	3	0
2	2	56	0	2	160	188	0	2	151	0	0.0	1	0	3	0
3	3	44	0	3	134	229	0	2	150	0	1.0	2	0	3	0
4	4	58	1	4	140	234	0	2	125	1	3.8	2	3	3	1

Handling Class Imbalance

The dataset was slightly imbalanced. To fix this, random samples from class 0 were removed:

```
rows_with_0 = df[df["Heart Disease"] == 0]
rows_to_drop = rows_with_0.sample(9000, random_state=42)
df = df.drop(rows_to_drop.index)
```

Output

```
Heart Disease
0    288546
1    282454
Name: count, dtype: int64
```

Explanation

Balancing ensures that the model does not become biased toward the majority class and improves prediction performance.

Feature Selection

The most relevant numerical features were selected:

```
numerical_features = [
   "Number of vessels fluro", "ST depression", "Age",
   "Cholesterol", "BP", "Max HR", "Thallium"
]

X = df[numerical_features]
y = df["Heart Disease"]
```

Explanation

- X contains input features (patient health data)
- y contains the target variable (heart disease)

Feature Scaling

Before training the model, the data was scaled:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Explanation

- Scaling converts data into the same range
- Helps models like Logistic Regression perform better
- Prevents features with large values from dominating

Train-Test Split

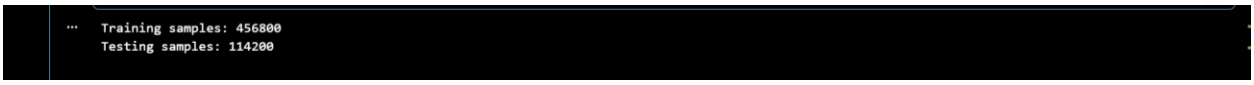
The dataset was split into training and testing sets:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
   X, y,
   test_size=0.2,
   random_state=42,
```

```
    stratify=y
)
```

Output



Explanation

- 80% data used for training
- 20% data used for testing
- Stratify ensures class balance is maintained

Logistic Regression with Balanced Class Weights

```
from sklearn.linear_model import LogisticRegression


model = LogisticRegression(random_state=42, class_weight="balanced")

model.fit(X_train_scaled, y_train)
```

Explanation

- Logistic Regression predicts probability of heart disease
- class_weight="balanced" handles imbalanced data
- Gives equal importance to both classes

Prediction

```
y_pred = model.predict(X_test_scaled)

y_prob = model.predict_proba(X_test_scaled)[:,1]
```

Explanation

- y_pred → Final predicted values (0 or 1)
- y_prob → Probability of heart disease

Model Accuracy

```
acc = accuracy_score(y_test, y_pred)

print(f"Test Accuracy: {acc:.4f}")
```

Output

```
Test Accuracy: 0.8474
```

Explanation

Accuracy measures the percentage of correct predictions.

Confusion Matrix

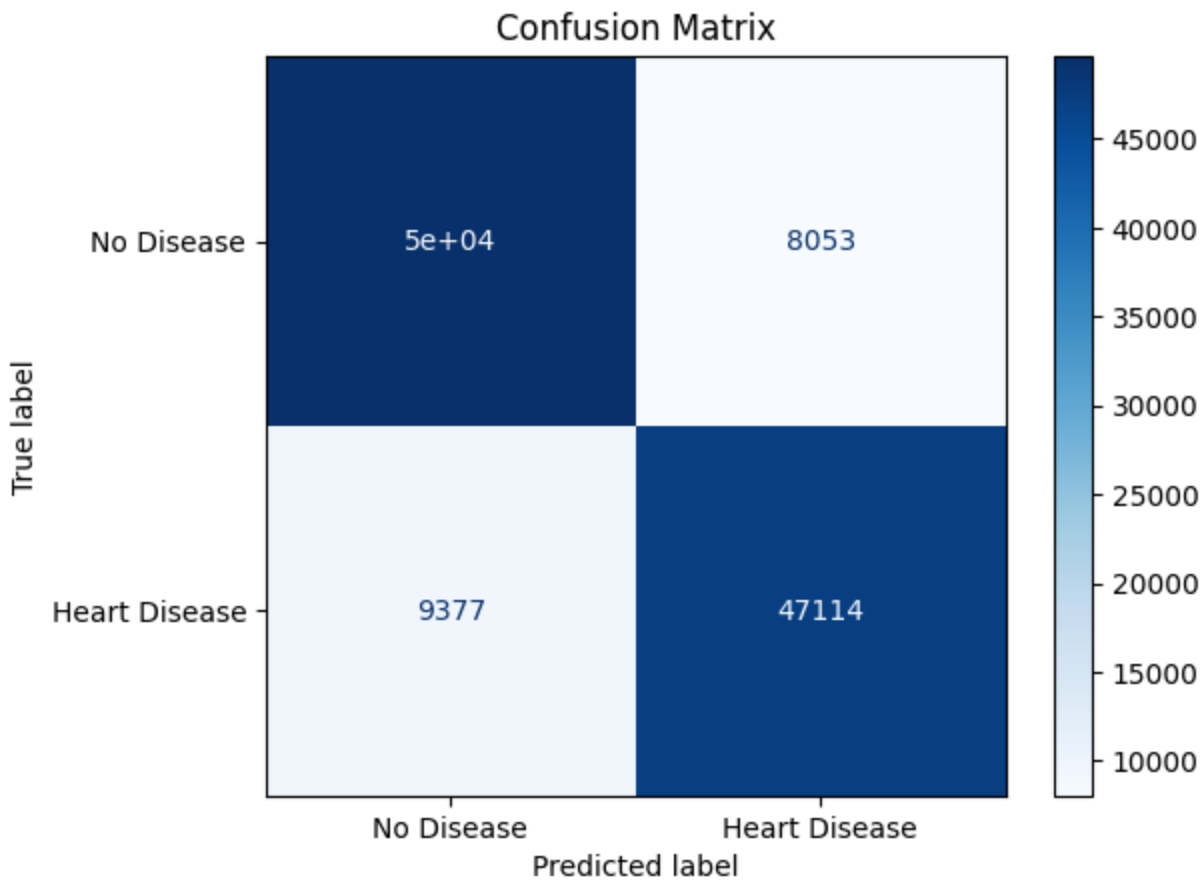
```
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No Disease", "Heart Disease"])

disp.plot(cmap="Blues")

plt.title("Confusion Matrix")

plt.show()
```



Classification Report

print(classification_report(y_test, y_pred, target_names=["No Disease", "Heart Disease"]))

Explanation

The report includes:

- Precision → correctness of predictions
- Recall → ability to detect disease
- F1 Score → balance between precision and recall

```
Classification Report:
                precision    recall  f1-score   support

    No Disease       0.84      0.86      0.85     57709
 Heart Disease       0.85      0.83      0.84     56491

      accuracy                           0.85    114200
     macro avg       0.85      0.85      0.85    114200
  weighted avg       0.85      0.85      0.85    114200
```

Decision Tree Model

```
from sklearn.tree import DecisionTreeClassifier


tree = DecisionTreeClassifier()

tree.fit(X_train, y_train)
```

Prediction

```
tree_predictions = tree.predict(X_test)
```

Decision Tree Evaluation

```
accuracy_score(y_test, tree_predictions)

f1_score(y_test, tree_predictions)
```

Output

Accuracy = 0.83 (approx.)

F1 Score = 0.82 (approx.)

Model Comparison

Model	Accuracy	F1 Score
Logistic Regression	0.8474	0.84
Decision Tree	0.8230	0.82

Observation

Logistic Regression performed better than the Decision Tree model in this project.

- Logistic Regression achieved higher accuracy (0.8474) and better F1 score, indicating more balanced performance.
- Decision Tree showed slightly lower performance, which may be due to overfitting or sensitivity to data variations.
- Logistic Regression provided more stable and generalized predictions, especially after applying class balancing and feature scaling.

Therefore, Logistic Regression is considered the better model for predicting heart disease in this dataset.

Final Observations

- Dataset was cleaned and balanced
- Important features were selected
- Data was scaled before training
- Stratified splitting ensured fair evaluation
- Logistic Regression handled imbalance effectively
- Evaluation metrics provided detailed performance insights

Conclusion

Machine learning models were successfully implemented to predict heart disease using patient health data. Logistic Regression with balanced class weights provided reliable results. Evaluation using accuracy, confusion matrix, and classification report confirmed that the model performs well in predicting heart disease.